

ՆԵՐԱԾՈՒԹՅՈՒՆ

FPGA-ները ճկուն, փոփոխվող սխեմաներ են, որոնք կարող են հեշտությամբ վերակազմավորվել նախագծողների կողմից: FPGA-ի վրա հիմնված դիզայնի կարևոր բաղադրիչ է CAD ծրագրային միջոցները, որոնք անհրաժեշտ են սխեմաները արդյունավետ օգտագործելու համար: Տիպիկ FPGA նախագծման գործընթացը բաղկացած է տրամաբանական սինթեզից (logic synthesis), տեխնոլոգիայի քարտեզագրումից (technology mapping), տեղաբաշխումից (placement) և երթուղավորումից (routing): Լավ տեղաբաշխուման ալգորիթմը ոչ միայն պետք է, տեղադրի միմյանց հետ կապակցված տարրերի միասին, այլ նաև պետք է համոզվի որ տրամաբանական տարրերը իրաի շատ մոտ չեն տեղադրված, որպեսզի հնարավոր լինի բոլոր տրամաբանական տարրերի միջև բոլոր հնարաոր կապերի հաստատումը և ապահովի շղթայի երթուղայնություն: Նաև պետք է հատկացնի մի շարք դատարկ բջիջներ՝ որպես տեղաբաշխման գործընթացի մաս՝ այդ բջիջներն երթուղաորման փուլում օգտագործելու համար:

Բացի այդ, սխեմայի տարրերի տարածական կազմակերպումը կարող է կանխորոշել միացման ընդհանուր չափսերն ու ձևը, ինչպես նաև արտադրական գործընթացի բարդությունն ու վերջնական արտադրանքի հուսալիությունը: Անհաջող նախագծված տեղաբաշխումը կարող է բերել մի շարք խնդիրների, ինչպիսիք են՝ ազդանշանների միջամտությունը, կապերի խաչաձևումը և էլեկտրամագնիսական ճառագայթումը, որոնք իրենց հերթին կարող են առաջացնել սարքի անսարքություններ կամ նույնիսկ ամբողջական խափանում:

Հետևաբար, տարրերի արդյունավետ տեղաբաշխումը միկրոէլեկտրոնիկայի սխեմաների մեջ առանցքային նշանակություն ունի ոչ միայն կատարողականի օպտիմալացման, այլ նաև նախագծման արդյունավետության բարձրացման և արտադրանքի երկարատև ու հուսալի շահագործման ապահովման համար: Ճիշտ նախագծված տեղաբաշխումը կարող է զգալիորեն նվազեցնել նախագծման ժամանակը, նվազեցնել արտադրական ծախսերը և բարձրացնել համակարգի

ընդհանուր կայունությունը:

Աշխատանքի նպատակն է մշակել ծրագիր, որի օգնությամբ կարելի է կատարել սխեմայի տարերի տեղաբաշխում, գենետիկ ալգորիթմի օգնությամբ, ըստ որոշ չափորոշիչների:

Աշխատանքի շրջանակներում շեշտը դրվելու է ուղղանկյուն երկրաչափական օբյեկտների վրա:

Այս ծրագրի իրականացման համար օգտագործվել է ծրագրավորման C++ լեզուն, Qt framework և «Boost C++ Libraries» գրադարանները:

ԽՆԴՐԻ ԴՐՎԱԾՔԸ

Պահանջվում է մշակել և իրագործել գենետիկ ալգորիթմների սխեմայի տարրերի տեղաբաշխման ծրագրային միջոց:

ԳԼՈՒԽ 1

ԳՐԱԿԱՆՈՒԹՅԱՆ ԱԿՆԱՐԿ

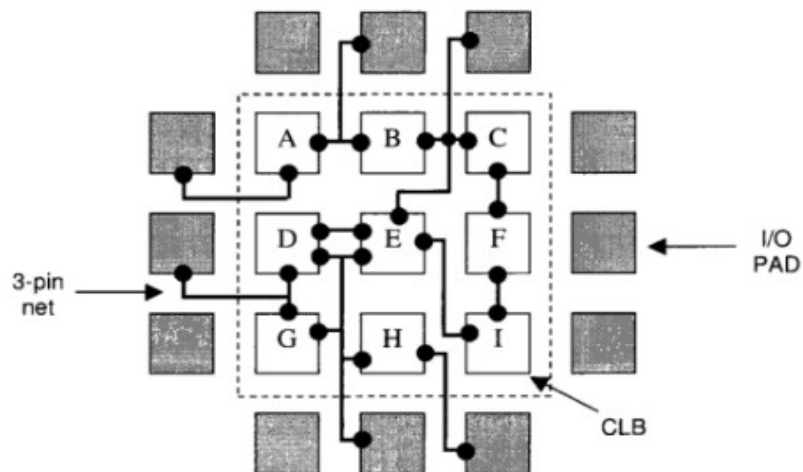
1.1 Տեղաբաշխման խնդրի ակնարկ

FPGA-ի տարրերի տեղադրման խնդիրը սովորաբար սկսվում է տրամաբանական բլոկների ցանցային ցուցակից(netlist) և դրանց փոխկապակցվածությունից: Տեղադրման արդյունքը թիրախային FPGA-ի վրա բոլոր բլոկների ֆիզիկական տեղադրումն է, որը նվազագույնի է հասցնում մեկ կամ մի քանի հատուկ ծախսային ֆունկցիաներ(cost function): Տեղաբաշխման խնդրի ֆորմալ նկարագրությունը հետևյալն է՝

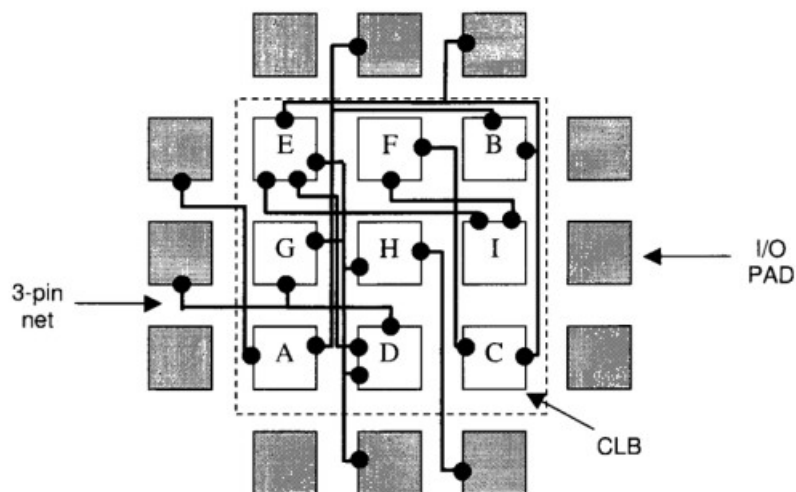
Given a set of blocks $B = \{b_1, b_2, \dots b_m\}$, a set of signals $S = \{s_1, s_2, \dots s_n\}$, and a set of locations on the field-programmable gate-array $L = \{l_1, l_2, \dots l_p\}$, where $p \geq |B|$. $\forall b_i \in B$, there is a set of signals $S_{b_i} \subseteq S$, and $\forall s_i \in S$, there is a set of blocks B_{s_i} , $B_{s_i} = \{b_j \mid s_i \in S_{b_j}\}$. B_{s_i} only contains all the blocks that send or receive signal s_i , and S_{b_j} only contains all the signals that are sent or received by block b_j . The goal is to assign each block $b_i \in B$ to a location $l_j \in L$ such that the chosen objective function is optimized[†].

Տարրերի տեղադրում կատարելիս ամենահիմնական նպատակն է նվազագույնի հասցնել երթուղին ավարտելու համար պահանջվող մետաղալարերի երկարությունը: Երթուղային ծախսերը(routing cost) օգտագործվում են, քանի որ դրա նվազեցումը նվազեցնում է մի շարք հարակից նախագծային պարամետրներ: Երթուղիների երկարությունը նվազեցնելով, բոլոր փոխկապակցումների համար պահանջվող երթուղային ռեսուրսները կրճատվում են: Տեղադրում կատարելիս ամենահիմնական նպատակն է նվազագույնի հասցնել երթուղին ավարտելու համար պահանջվող մետաղալարերի երկարությունը: Երթուղային ծախսերը օգտագործվում են, քանի որ դրա նվազեցումը նվազեցնում է մի շարք հարակից նախագծային արամետներ: Սա հանգեցնում է շղթայի արագագործության ավելացմանը միացման հզորության և դիմադրության նվազման շնորհիվ: Էլեկտրաէներգիայի սպառումը, որը ևս մեկ կարևոր

պարամետր է FPGA-ի իրականացման որակը չափելու համար, նույնպես կրճատվում է:



Նկար 1.1 Տեղաբաշխման լավ օրինակ



Նկար 1.2 Տեղաբաշխման վատ օրինակ

1.2 Սխեմայում տարրերի տեղադրման մեթոդներ

Վերջին մի քանի տարիների ընթացքում սխեմայում տարրերի տեղադրման շատ

ալգորիթներ են առաջարկվել՝ կարգավորելու լարերի երկարությունը նվազագույնի հասցնելու նպատակը: Այնուամենայնիվ, քանի որ տեղաբաշխման խնդիրը NP-hard է, հայտնի չէ, որևէ բազմանդամային ժամանակում տեղաբաշխման խնդիրն ճշգրիտ լուծում տվող ալգորիթ: Հետևաբար, ալգորիթների մեծ մասը էվրիստիկ են, որոնք ձգտում են գտնել «լավ» լուծումներ «ողջամիտ» ժամանակում: Այս մեթոդները կարելի է պայմանականորեն բաժանել հինգ հիմնական կատեգորիայի .՝

- 1) Բաժանման վրա հիմնված տեղաբաշխում (partitioning-based placement)
- 2) Մոդելավորված սառեցման (simulated-annealing) վրա հիմնված տեղադրում,
- 3) Բազմաստիճան խմբավորում (multilevel clustering),
- 4) Վերլուծական հիմքով տեղաբաշխում: (analytical-based placement)
- 5) Էվոլյուցիոն տեղաբաշխում (Evolutionary placement)

FPGA-ի տեղադրման համատեքստում չափազանց թանկ է որոշել երթուղային ռեսուրսների ճշգրիտ կոնֆիգուրացիան, որն անհրաժեշտ է բլոկների միջև ֆիզիկական կապեր իրականացնելու համար, ինչը ևս մեկ NP-դժվար խնդիր է: Բացի միացումների միջև հեռավորությունից (լարի երկարությունից), կարող են լինել լարերի հարթակի վրա լարերի քանակի, որոշակի լարերի թույլատրելի երկարության կամ պտույտների քանակի, երթուղային հանգույցների առկայության և այլնի սահմանափակումներ: Այդ իսկ պատճառով երթուղային արժեքը(routing cost) մոտավոր է ընտրվում տեղաբաշխման ժամանակ:

1.2.1 Բաժանման վրա հիմնված տեղաբաշխում

Բաժանման վրա հիմնված տեղադրման մեթոդները հայտնի են նաև որպես նվազագույն կտրվան (min-cut) մեթոդներ: Հիմնական գաղափարն այն է, որ օգտագործվում է գրաֆի բաժանման ալգորիթ՝ FPGA-ի տարածքը երկու մասի բաժանելու համար: Այնուհետև կիրառվում է սխեմայի բաժանման ալգորիթ՝ որոշելու համար, թե որ տրամաբանական բլոկը որ հատվածում պետք է տեղակայվի՝ նպատակ ունենալով նվազեցնել երկու բաժինների սահմանը հատող ցանցերի կտրվածքների

քանակը և բարձր փոխկապակցվածությամբ բլոկները նույն բաժնի մեջ տեղավորել: Այս գործընթացները կրկնվում են ռեկուրսիվ կերպով, մինչև յուրաքանչյուր բաժնում մնան ընդամենը մի քանի բլոկ: Բաժանման վրա հիմնված տեղադրման ալգորիթմների առավելությունն այն է, որ դրանք շատ արագ են աշխատում: Քանի որ դրանք օգտագործում են բաժանման և նվաճման (divide-and-conquer) ռազմավարություն՝ մեծ խնդիրները փոքր խնդիրների բաժանելու միջոցով, դրանք զգալիորեն նվազեցնում են որոնման տարածքը: Սակայն, քանի որ կտրվածքի չափը ճշգրիտ կերպով չի արտացոլում մալուխների երկարությունը, ժամանակային սահմանափակումները կամ երթուղիների հնարավորությունը, արդյունքի որակը այնքան էլ բարձր չէ համեմատած այլ տեղադրման մեթոդների հետ:

1.2.2 Մոդելավորված այրում (simulated annealing)

Մոդելավորված այրումը (SA) լայնորեն կիրառվում է համակցված օպտիմալացման խնդիրների լուծման համար և հաջողությամբ կիրառվել է սխեմաների տեղադրման ժամանակ (ASIC նախագծման համատեքստում): Ինչպես անունն է հուշում, այս մեթոդը նմանակում է հալեցված մետաղի աստիճանաբար սառեցման գործընթացը՝ լավ բյուրեղային կառուցվածք ստանալու նպատակով: Իդեալում սառեցված բյուրեղը պետք է լինի ամենացածր էներգետիկ վիճակում, որը համակցված օպտիմալացման խնդրում համապատասխանում է գլոբալ օպտիմալ կազմաձևին: SA-ի վրա հիմնված տեղադրման ալգորիթմները հեշտությամբ հարմարեցվում են՝ հաշվի առնելու ցանկացած հայտնի սահմանափակում և օպտիմալացման նպատակ: Բացի բարձրացման (hill-climbing) հատկությունից, այս մեթոդի առավելությունն այն է, որ այն կարող է ընդունել նաև ոչ բարելավող քայլեր՝ թույլ տալով դուրս գալ տեղական օպտիմումներից:

Մոդելավորված սառեցումը պատկանում է ստոխաստիկ որոնման ալգորիթմների դասին, որոնք ընդունում են ցանկացած պատահականորեն հանդիպող լուծում, որը գտնվում է ներկայումս դիտարկվող լուծումների հարևանության մեջ՝ որոշակի

հավանականությամբ:

Գործնականում նոր հարևան լուծումները ստեղծվում են ներկայիս լուծումից՝ ինկրեմենտալ ձևով: Եթե նոր լուծման արժեքը (հաշվարկված համապատասխան նպատակային ֆունկցիայով) նվազում է, ապա այն ընդունվում է: Բայց եթե նոր լուծումը ավելի վատ արժեք ունի, այնուամենայնիվ կարող է ընդունվել՝ հավանականությամբ $e^{(-\Delta C/T)}$, որտեղ ΔC -ը արժեքի փոփոխությունն է, իսկ T -ն համարժեք է ջերմաստիճանին մետաղի բյուրեղացման գործընթացում:

Պարամետր T -ն վերահսկում է ոչ բարելավող քայլերի ընդունման հավանականությունը: Բարձր T արժեքը որոնումը դարձնում է պատահական, իսկ ցածր T արժեքը որոնումը վերածում է սովորական բարձրացող մեթոդի: Ուստի անհրաժեշտ է գտնել տվյալ խնդրի համար համապատասխան T արժեք (որոնման ընթացքում):

T -ի փոփոխման արագությունը կոչվում է **սառեցման ժամանակացույց (annealing schedule)**, և այն մեծ ազդեցություն ունի վերջնական լուծման որակի և աշխատանքային ժամանակի վրա: Սկզբում T -ն սահմանվում է բարձր արժեքով, որպեսզի մեծամասնությունը ոչ բարելավող լուծումների ընդունելի լինի: Սակայն գործընթացի ընթացքում T -ն աստիճանաբար նվազում է (նմանեցնելով սառեցմանը), նվազեցնելով վատ լուծումների ընդունման հավանականությունը: Որոնման վերջնական փուլերում T -ն դառնում է իր սկզբնական արժեքի փոքր մասնաբաժին, և հիմնականում ընդունվում են միայն բարելավող լուծումները:

SA ալգորիթի առանձնահատկությունն այն է, որ այն կարողանում է դուրս գալ տեղական օպտիմումներից, ինչը շատ այլ որոնման մեթոդներ չեն կարողանում:

Ինչպես բոլոր որոնման մեթոդները, մոդելավորված սառեցումն ունի թե՛ առավելություններ, թե՛ թերություններ: Առավելությունն այն է, որ տեսական վերլուծությունները [56] ցույց են տալիս՝ որոշ պայմանների դեպքում (որոնման երկարություն յուրաքանչյուր T -ի ժամանակ և T -ի թարմացման կանոններ) այն համայն հավանականությամբ կոնվերգում է դեպի գլոբալ օպտիմալ լուծում: Բացի այդ, նոր

օպտիմալացման նպատակներ կամ սահմանափակումներ ավելացնելը շատ ավելի հեշտ է SA-ում՝ համեմատած այլ որոնման մեթոդների հետ: Սակայն գործնականում քիչ տեղեկատվություն կա այն մասին, թե ինչպես պետք է ընտրվեն ճիշտ պարամետրերը կոնկրետ իրագործման համար: Բացի այդ, գլոբալ օպտիմում գտնելու ժամանակը կարող է չափազանց երկար լինել: Արդյունքում՝ SA-ի ներկայիս կիրառությունների մեծ մասը օգտագործում են պարզ, սակայն արդյունավետ մոտեցումներ՝ լավ՝ ոչ իդեալական լուծումներ ստանալու համար:

1.2.3 Մոդելավորված այրում (simulated annealing)

Անալիտիկ ալգորիթները համարվում են ամենահեռանկարային մեթոդներից մեկը՝ արագ տեղադրման խնդիրներ լուծելու համար: Այս ալգորիթները մոտենում են խնդիրն վերևից դեպի ներքև՝ հաշվի առնելով գլոբալ կապերը՝ ի տարբերություն փոքր մասշտաբով բազմաթիվ փոփոխությունների գնահատման:

Այս մոտեցման մեջ ներառված են ինչպես **ուժային մեթոդները** (*force-directed*) այնպես էլ **քառակուսային ծրագրավորման** (*quadratic programming*) մեթոդները:

Ուժային մեթոդը ներմուծում է ձգող, վանող և այլ լրացուցիչ ուժեր, ապա այդ ուժերի հիման վրա լուծում է գծային հավասարումների համակարգ:

Քառակուսային ծրագրավորման մեթոդը լուծում է տեղադրման խնդիրը՝ փոփոխելով փոփոխություններ քառակուսի ծրագրավորման խնդիրներ, որոնք ստացվում են սխեմայի կապային կառուցվածքից: Այս մեթոդը պահպանում է տեղադրման խնդիրների ամբողջական տեսլականը, ուստի հաճախ օգտագործվում է որպես գլոբալ օպտիմալացման մեթոդ:

Ընդհանուր առմամբ, քառակուսի ծրագրավորման մեթոդները որպես մուտք ստանում են հիպերգրաֆային ցանցի նկարագրություն (*netlist*) և ձգտում են նվազեցնել ընդհանուր քառակուսի հաղորդման երկարությունը: Նրանց նպատակային ֆունկցիան արտահայտվում է հետևյալ կերպ՝

$$\Phi(x, y) = \frac{1}{2} \sum_{i,j} W_{ij} [(x_i - x_j)^2 + (y_i - y_j)^2]$$

Այստեղ x, y -ը տրամաբանական բլոկի կոորդինատներն են: $W_{i,j}$ -ը քաշն է այն կապի, որը միացնում է բլոկ (x_i, y_i) -ը բլոկ (x_j, y_j) -ին: Քանի որ երկու բլոկ կարող են միացված լինել մեկից ավելի ցանցերով (net), հիպերգրաֆը նախ պետք է փոխարկվի քաշավորված գրաֆի: Այս փոխարկման համար կիրառվում են երկու մոդել՝ կլիկի մոդել (clique model) և աստղաձև մոդել (star model): Այս փոխակերպումները հնարավորություն են տալիս հիպերգրաֆից անցում կատարել սովորական քաշավորված գրաֆի, որի վրա արդեն կարելի է կիրառել քառակուսի օպտիմալացման ալգորիթները՝ տեղադրման (placement) խնդիրը լուծելու նպատակով:

1.2.3 Բազմաստիճան խմբավորում (simulated annealing)

Քանի որ FPGA-ների տրամաբանական հզորությունն ու ֆունկցիոնալությունը շարունակաբար աճում են, նույնը տեղի է ունենում նաև դրանց վրա քարտեզագրվող նախագծերի (շղթաների) բարդության հետ: Մեծ նախագծերի տեղադրման (placement) բարդությունը նվազեցնելու ժամանակակից մեթոդներից մեկը բազմաստիճան ուղղվարությունն է (*multilevel strategy*): Բազմաստիճան մեթոդը կառուցում է խնդիրների հիերարխիա՝ ներքևից վեր (*bottom-up*)՝ աստիճանաբար խմբավորելով՝ դարձնելով խնդիրը ավելի պարզեցված: Այնուհետև վերևից ներքև (*top-down*) իրականացվում է բարելավումների փոխանցում այս հիերարխիայի մակարդակներում, մինչև ի վերջո լուծվում է խնդիրը իր սկզբնական՝ ամենանուրբ մակարդակում:

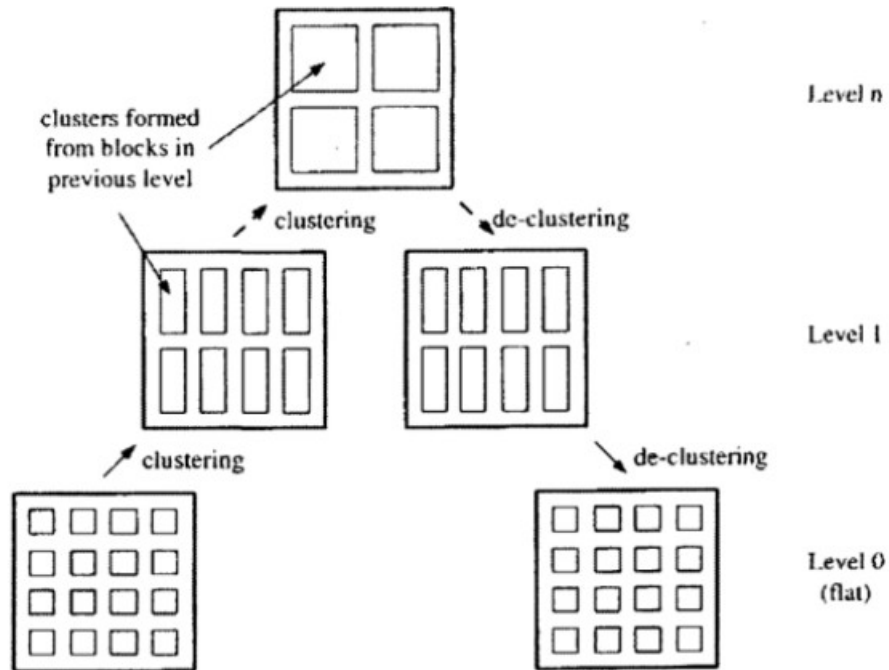
Բազմաստիճան մոտեցման ընթացքը ունի երկու հիմնական փուլ՝

1. Ներքևից վեր – խմբավորում (clustering)

Այս փուլում **խիստ կապված բլոկները խմբավորվում են կլաստերների (clusters)** մեջ՝ դարձնելով խնդիրը կոպիտ մոտեցմամբ ներկայացված:

2. Վերևից ներքև – տեղադրում

Այս փուլում տեղադրվում են կլաստերները՝ օգտագործելով ժամանակատար ալգորիթներ (օրինակ՝ simulated annealing), քանի որ խնդիրը այժմ պարզեցված է:



Նկար 1.3 Բազմաստիճան խմբավորում

Այնուհետև կիրառվում է **դեկլաստավորման (declustering)** փուլ, որի ընթացքում վերականգնվում է սկզբնական FPGA դասավորությունը՝ ելնելով կլաստերների նախնական տեղադրման արդյունքներից: Այս ընթացքում, «բացված» բլոկները պետք է հնարավորինս մոտ լինեն իրենց **ծանրության կենտրոնին (center-of-gravity)**:

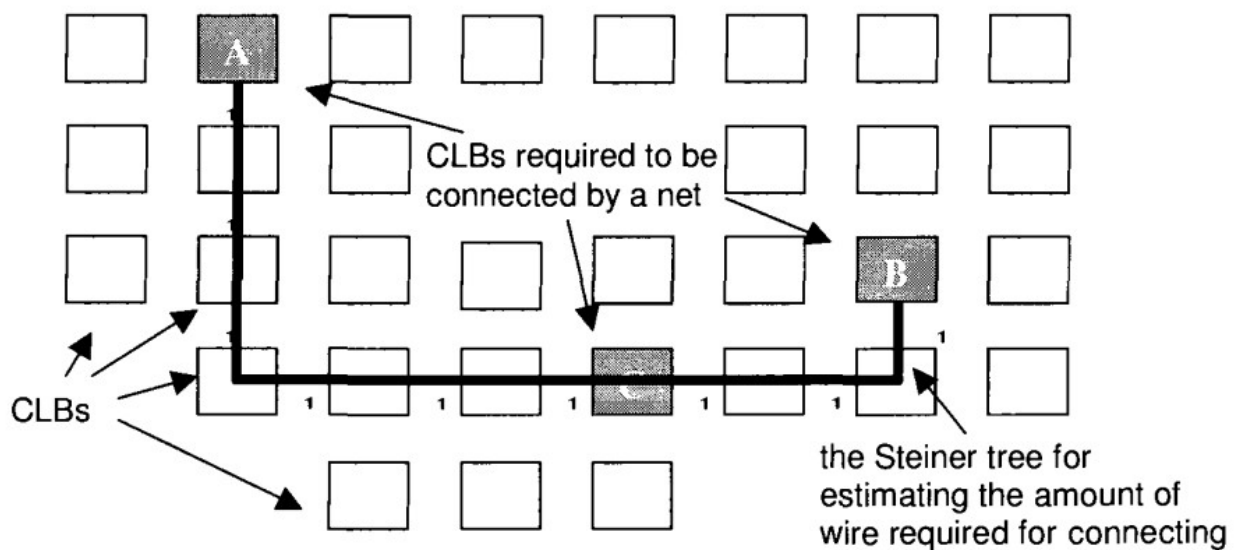
2.4 Մալուխի երկարության հաշվարկման մոդելներ

Քանի որ տվյալ տեղադրման համար մալուխի ճշգրիտ երկարությունը կարելի է իմանալ միայն երթուղավորուման(routing) փուլից հետո, FPGA-ի տեղադրման ալգորիթները պահանջում են մալուխի երկարության ճշգրիտ և արագ հաշվարկվող

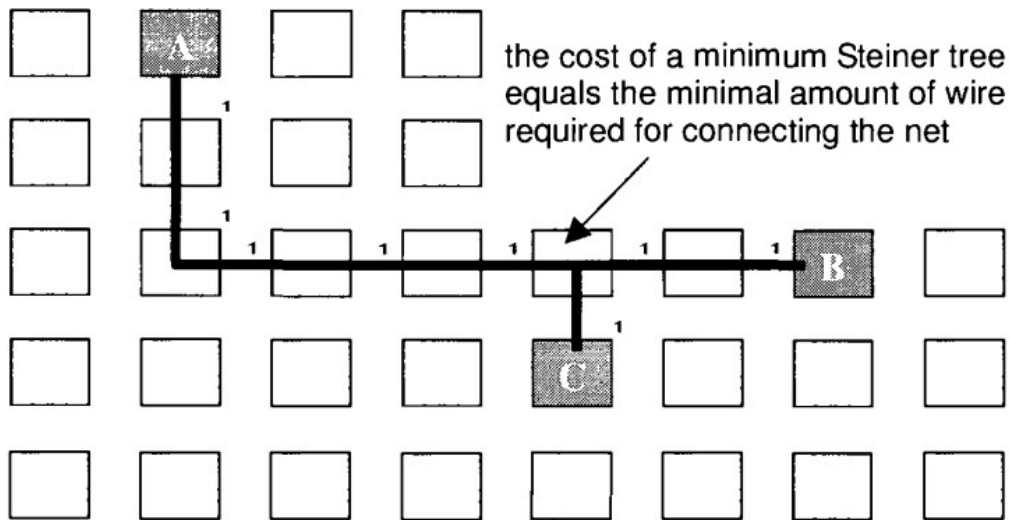
գնահատականներ: Հիմնական մոդելներն են՝ մինիմալ Սթայներ ծառ, մինիմալ Սթայներյան ծառ (**Minimum Steiner Tree**), կիսա-պարագծային մալուխի երկարություն(**Half-Perimeter Wire Length, HPWL**), աստղաձև (Star Model) և այլ: Ստորև ներկայացվում է այս մոդելների համառոտ նկարագրությունը:

1. Մինիմալ Սթայներյան ծառի մոդել՝

Մինիմալ Սթայներյան ծառը նվազագույն կմախքային ծառ է, որը միացնում է տերմինալների որոշակի բազմություն: Մինիմալ Սթայներյան ծառի ցանցի մոդելում կարգավորելի տրամաբանական բլոկների (CLB) յուրաքանչյուր մուտքային կամ ելքային պին համապատասխանում է տերմինալի: Ցանցի համար անհրաժեշտ մետաղալարի երկարությունը նվազագույնի հասցնելը՝ համարժեք է մինիմալ Սթայներյան ծառի գտնելու խնդրին: Նկար 1.4-ում և 1.5-ում ներկայացված են երեք տերմինալանոց ցանցեր:



Նկար 1.4 Սթայներ ծառ, որի ընդհանուր մետաղալարի երկարությունը հավասար է 9-ի:

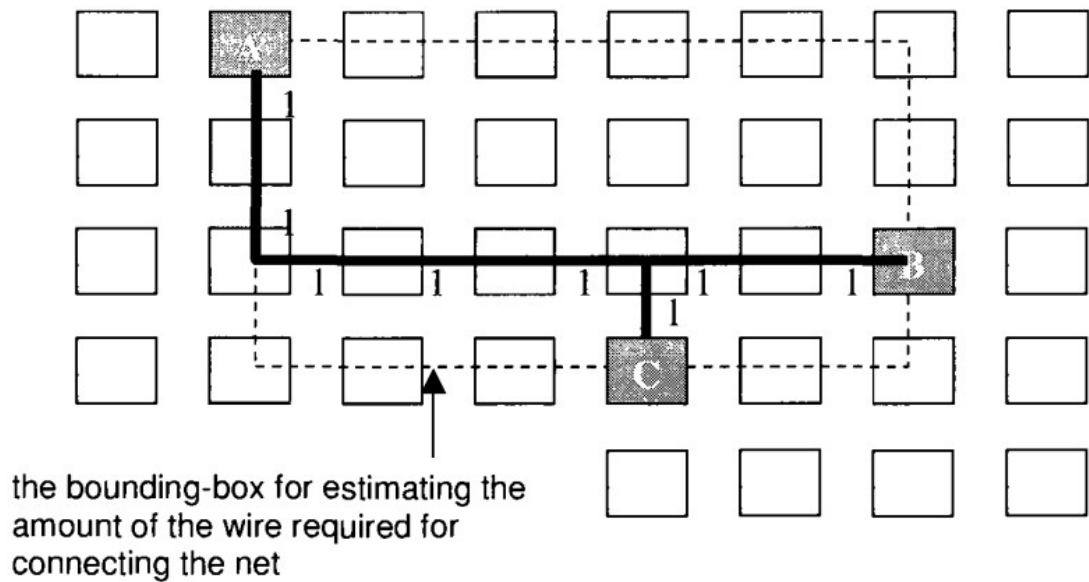


Նկար 1.5 Սթայներ ծառ, որի ընդհանուր մետաղալարի երկարությունը հավասար է 8-ի:

2. Կիսա-պարագիծային լարային երկարության մոդել՝

Ամենաշատ կիրառվող լարային երկարության մոդելը, կոչվում է Կիսա-պարագիծային լարային երկարության մոդել (HPWL): HPWL-ը գնահատում է լարային երկարությունը՝ որպես այն փոքրագույն ուղղանկյան կիսա-պարագիծը, որը պարունակում է ցանցում ընդգրկված բոլոր բլոկները: Նկար 1.6-ում պատկերված է ցանց, որը միացնում է երեք բլոկ՝ A, B և C: Կետագծերով հշված ուղղանկյունը ցանցը պարփակող փոքրագույն ուղղանկյունն է, որի պարագիծը հավասար է 16-ի: Սթայներյան ծառը ներկայացված է հոծ գծով որի ընդհանուր մետաղալարի երկարություն 8, ինչը հենց ուղղանկյան պարագծի կեսն է:

Ակնհայտ է, որ HPWL մոդելը ճշգրիտ է ցանցերի համար, որոնք միացնում են երկու կամ երեք բլոկ, սակայն այն թերի է այն ցանցերի համար որոնք միացնում են ավելի քան երեք բլոկ:



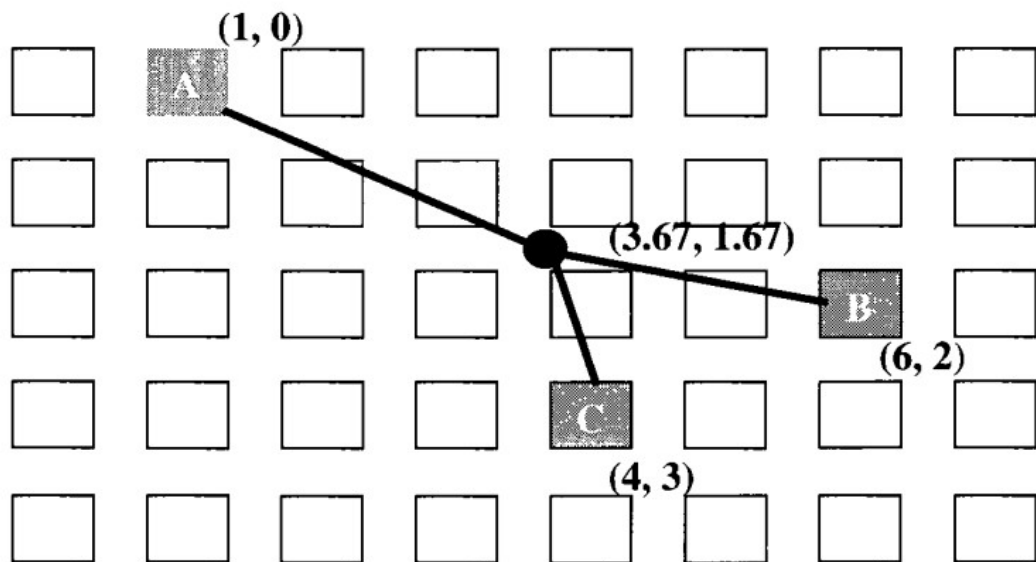
Նկար 1.6 Սթայներ ծառ, որի ընդհանուր մետաղալարի երկարությունը հավասար է 8-ի:

3. Աստղաձև մոդել՝

Աստղաձև մոդելը ավելացնում է նոր գագաթ ձգողականության կենտրոնում և ներկայացնում է սկզբնական ցանցը կողերով, որոնք միացնում են կենտրոնը նախապես գոյություն ունեցող գագաթներին: Գագաթ A-ի և կենտրոնի միջև կողը նշվում է որպես E_A : Յուրաքանչյուր E_A կող ունի երկարություն L_A , որը հավասար է A-ի և կենտրոնի միջև հեռավորությանը: Ցանցի մետաղալարերի երկարությունը գնահատվում է՝

$$\sum_{\text{edge} \in \text{net}} 2^{L_{\text{edge}}}$$

Լարերի երկարությունը x և y տարածություններում հաշվարկվում է առանձին: Աստղաձև մոդելը ցուցադրված է Նկ. 1.7-ում:



Նկար 1.7 Աստղաձև մոդելը Նկ. 1.4 ում պատկերված ցանցի համար:

Այս բոլոր մոդելները նպատակ ունեն արագ և բավարար ճշգրտությամբ գնահատել մալուխի երկարությունը մինչև վերջնական երթուղայնացումը (routing)՝ որպեսզի հնարավոր լինի ավելի արդյունավետ կատարել տեղադրման (placement) փուլը:

ԳԼՈՒԽ 2

ԸՆԴՀԱՆՈՒՐ ՏԵՍԱԿԱՆ ԴՐՈՒՅԹՆԵՐ

2.1 Գենետիկ ալգորիթմների կիրառումը տեղաբաշխման խնդրում

Գենետիկ ալգորիթմները տարբերվող մոտեցում են տեղաբաշխման խնդրում: Մինչ այլ ալգորիթմները կրկնվող կերպով բարելավում են տեղաբաշխումը՝ փոխանակելով երկու տարր կամ տեղափոխելով մեկ տարր, գենետիկ ալգորիթմները աշխատում են սկզբնական տեղաբաշխման մի շարքով, որոնք ներկայացնում են սկզբնական **պոպուլյացիան**: Պոպուլյացիայի յուրաքանչյուր անհատ **սիմվոլների** շարան է: Սիմվոլներին են անվանում են **գեներ**, իսկ գեների շարանը կոչվում է **քրոմոսոմ**: Քրոմոսոմը ներկայացնում է տեղաբաշխման խնդրի լուծումը: Գենների մի շարք, որոնք կազմում են մասնակի լուծում, կոչվում է **սխեմա**: Ալգորիթմը փորձում է միավորել երկու տարբեր տեղաբաշխումների մասնակի լուծումները՝ ավելի լավ տեղաբաշխում ստանալու համար:

Գենետիկական ալգորիթմի հիմնական գաղափարները ոգեշնչվել են կենսաբանական էվոլյուցիոն գործընթացից: Գենետիկական ալգորիթմները բազմիցս իրականացնում են վերարտադրման-գնահատման ցիկլը հետևյալ կերպ: Այս ցիկլի յուրաքանչյուր կրկնության ընթացքում (հայտնի է որպես սերունդ) ընթացիկ պոպուլյացիայի անհատները գնահատվում են՝ օգտագործելով ֆիզիկական հարմավորականության որոշ չափանիշներ: Հարմավորականությունը կապված է ծախսերի ֆունկցիայի հետ, իսկ նվազագույնի հասցնելու խնդրի դեպքում սովորաբար ծախսերի ֆունկցիայի հակառակն է: Ֆիթնես արժեքի հիման վրա **պոպուլյացիայ** միջից ընտրվում են միաժամանակ երկու անհատներ (կոչվում են ծնողներ), ընդ որում ավելի պիտանի անհատները ընտրվելու ավելի մեծ հավանականություն ունեն: Այնուհետև ընտրված ծնողների վրա կիրառվում են մի շարք գենետիկական օպերատորներ՝ սերունդներ կոչվող նոր անհատներ(**offsprings**) ստեղծելու համար, որը

հայտնի է որպես վերարտադրություն(**reproduction**): Այս գենետիկ օպերատորները համատեղում են երկու ծնողների առանձնահատկությունները: Անկախ մոդելից, ձևայնացված (ֆորմալ) տեսքով գենետիկական ալգորիթմը կարող է ներկայացվել հետևյալ 9 պարամետրի բազմությամբ: **GA = (P⁰, n, n_i, m, S, C, M, f, t)** որտեղ՝

P⁰ - սկզբնական պոպուլյացիան է,

n - պոպուլյացիայի չափը,

m - առանձնյակի (ինդիվիդումի) ձևայնացված ներկայացման (կոդի) երկարությունն է (քրոմոսոմի երկարությունը),

S - ընտրության (սելեկցիայի) օպերատոր,

C - խաչասերման օպերատոր,

M - մուտացիայի օպերատոր,

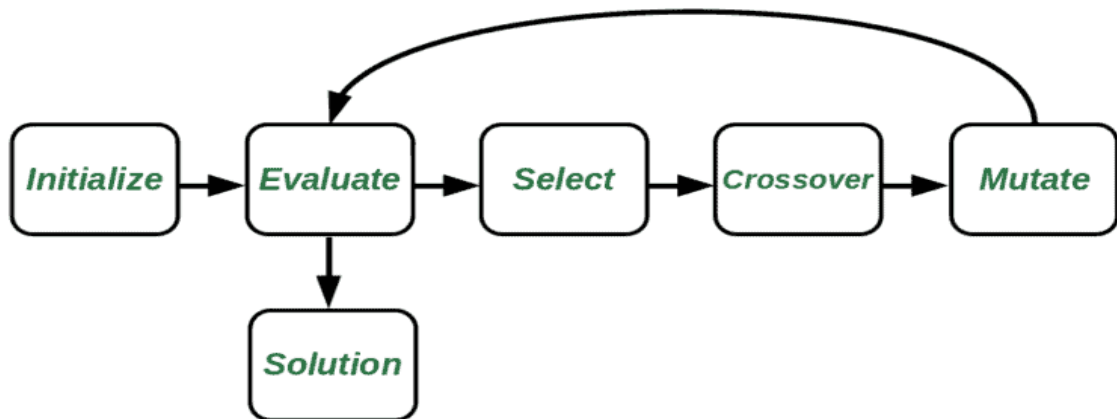
f - համապատասխանության ֆունկցիա (ֆիտնես ֆունկցիա, որակի չափանիշ),

t - կանգառի պայմանը:

Դասական գենետիկ ալգորիթմը ենթադրում է հետևյալ քայլերը՝

- 1) m երկարությամբ վեկտորներից P⁰ սկզբնական պոպուլյացիայի գեներացում, որը բաղկացած է n առանձնյակներից:
- 2) C խաչասերման օպերատորի կիրառում ընտրված ծնողների համար:
- 3) M մուտացիայի օպերատորի կիրառում ժառանգների համար:
- 4) S ընտրության (սելեկցիայի) օպերատորի կիրառում 2, 3 և 4 քայլերի արդյունքում ձևավորված առանձնյակներից առանձնացվում են այն n ամենաապիտանի առանձնյակները, որոնք համաձայն f համապատասխանության ֆունկցիայի պետք է անցնեն հաջորդ սերունդ:
- 5) Հերթական սերնդի պոպուլյացիայի ձևավորում:
- 6) 2-ից 6 քայլերի բազմակի կրկնություն մինչև ալգորիթմի կանգառի

պայմանը:



Նկար 1.4 Գենետիկ ալգորիթմի քայլերը

Որպես ալգորիթմի կանգառի t պայման կարող է հանդես գալ՝

- f համապատասխանության ֆունկցիայի իմաստով գլոբալ կամ ընդունելի լոկալ տարբերակի ստացում
- Սերունդների նախորոք որոշված քանակի գեներացում
- Խնդրիլ լուծմանը հատկացված ժամանակի ավարտ

2.2 Գենետիկ օպերատորներ

2.2.1 Խաչասերում

Խաչասերումը հիմնական գենետիկ օպերատորն է: Այն գործում է երկու ծնող անհատների վրա և գեներացնում է նոր սերունդ: Եթե Π_a և Π_b երկու թեկնածու քրոմոսոմ են, որոնք ներկայացնում են վավեր տեղաբաշխումներ, ապա Խաչասերման գործողությունը սահմանվում է որպես $\Pi_a \times \Pi_b \rightarrow \Pi_o$, որտեղ Π_o -ն, մեկ այլ վավեր տեղաբաշխում է կամ այլ կերպ ասված քրոմոսոմ: Խաչասերումը համատեղում է երկու ծնողների սխեմաները, հետևաբար սերունդը ժառանգում է ծնողների որոշ առանձնահատկություններ: Խաչասերման ամենապարզ ձևը բաղկացած է ծնողների գենային տողերում պատահական կտրվածքի կետ ընտրելուց և սերունդ ստեղծելուց՝

մի ծնողի հատվածը կտրված կետից ձախ կողմում մյուս ծնողի հատվածը կտրվածքի աջ կողմում: Օրինակ՝ ABCD և BDCA երկու տողերի միջև պարզ խաչասերման արդյունքը կարող է լինել ABCA և BDCD նոր տողեր: Եթե յուրաքանչյուր նիշ համապատասխանում է տեղաբաշխման խնդրի բջիջին, ապա վավեր տողը պետք է ունենա յուրաքանչյուր նիշի մեկ և միայն մեկ օրինակ: Գրականության մեջ, խաչասերման մեխանիզմները արժանացել է ամենամեծ ուշադրությանը: Ամենահայտնի մեխանիզմներն են՝ մեկ կետանոց խաչասերում, բազմակետ խաչասերում, կոմբինացված խաչասերում, հավասարաչափ խաչասերում, մասնակի համընկնմամբ փոխանակում, կարգավորված փոխանակում և այլն: Կիրառվում են նաև այդ մեխանիզմների տարբեր մոդիֆիկացիաներ:

Ստորև բերված են այդ մեխանիզմների մի քանի օրինակներ:

1) Մեկ կետային

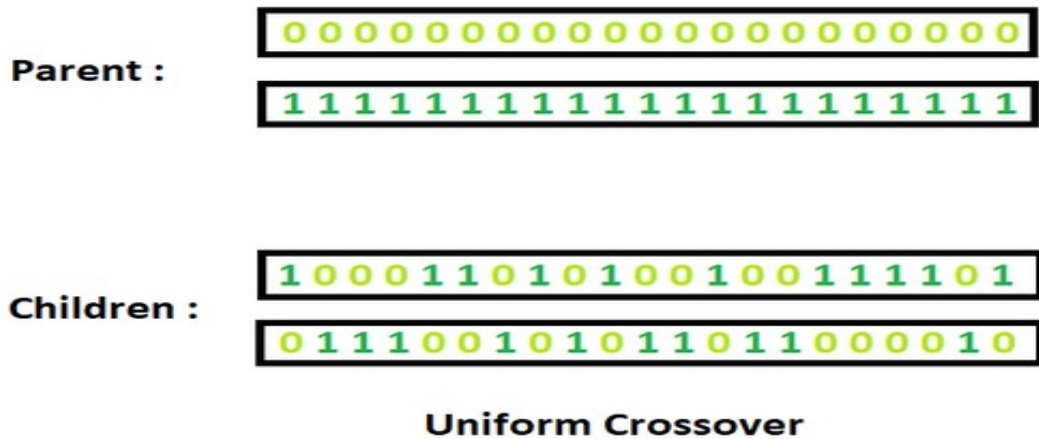
Chromosome1	11011 00100110110
Chromosome2	11011 11000011110
Offspring1	11011 11000011110
Offspring2	11011 00100110110

Նկար 1.4 Մեկ կետային խաչասերման վիզուալ ներկայացումը

Յուրաքանչյուր ժառանգի առաջին կեսը կրկնում է համապատասխան ծնողի գեները, իսկ երկրորդ կեսում ներառվում են մյուս ծնողի միայն այն գեները, որոնք առաջին կեսում չկան, այնուհետև երկրորդ կեսում դատարկ մնացած գեները վերցվում են այդ ծնողի այն գեները որոնք դեռ չեն հանդիպել և լրացվում են պահպանելով իրենց փոխադարձհարաբերական դասավորությունը: Օրինակ՝ Ժառանգ 1 - ի երկրորդ առաջին կեսում վերցված են 1 - ին ծնողի մեկից մինչև 5 - րդ գենը ներառյալ իսկ երկրորդ կեսում

երկրորդ ծնող 6 - թ դեղից մինչև 11 - թ դեղնը ներառյալ: 2 - թ ժառանգի մոտ նույն տրամաբանությամբ ուղակի ծնողների տեղերը փոխելով:

2) Բազմակետ խաչասերում



Նկար 1.5 Բազմակետ խաչասերման վիզուալ ներկայացումը

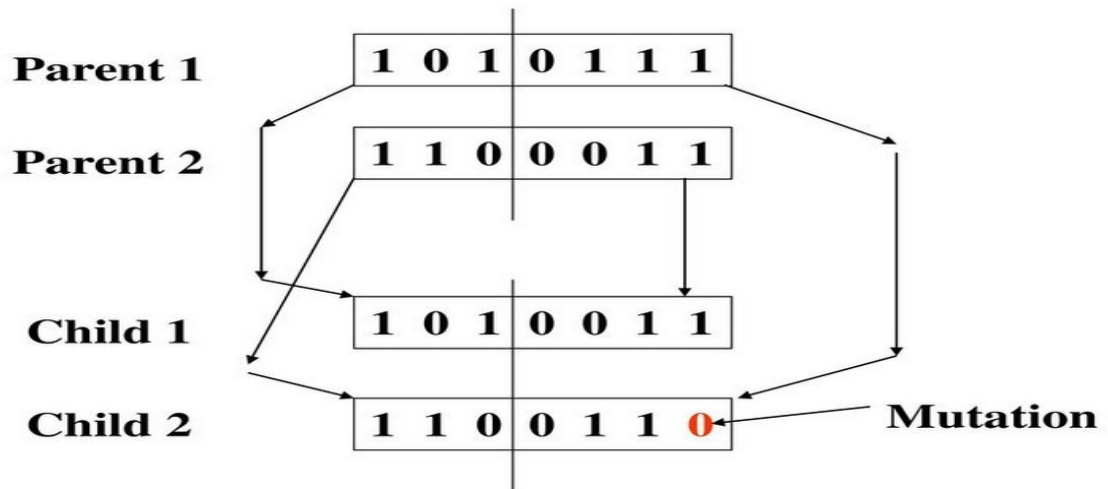
Այս դեպքում ծնողի քրոմոսոմը բաժանվում է M մասի և այդ մասերի, մի կեսը (օրինակ կենտ համարներով մասերը) փոխանցվում են ժառանգին, իսկ ժառանգի, ազատ մնացած գեները լրացվում են մյուս ծնողից մեկ կետային խաչասերմանը համաման ձևով:

2.2.2 Մուտացիա

Մուտացիան առաջացնում է պատահական փոփոխություններ սերունդների մեջ, որոնք առաջանում են խաչասերման օպերատորից հետո: Մուտացիան վերահսկվում է պարամետրով, որը կոչվում է մուտացիայի աստիճան: Մուտացիայի գործընթացը թույլ կտա պահպանել պոպուլյացիայի բազմազանությունը գենետիկ ալգորիթմի հետագա փուլերում: Մուտացիան նաև օգնում է խուսափել գենետիկ ալգորիթմին տեղային մինիմումից:

Մուտացիան վերահսկվում է պարամետրով, որը կոչվում է մուտացիայի աստիճան M_r . Յաճախ M_r նշանակում է, որ նոր գեների փոփոխությունը շատ ցածր է: Բարձր M_n - ը կհանգեցնի նրան, որ սերունդները կկորցնեն նմանությունը իրենց

ծնողներին: Մուտացիայի գործընթացը թույլ կտա պահպանել բնակչության բազմազանությունը գենետիկ ալգորիթմի հետագա փուլերում: Գրականությունում խորհուրդ է տրվում մուտացիայի աստիճանը ֆիքսել 0,001-ից 0,5-ի սահմաններում:



**Նկար 1.6 Մուտացիա օպերատորի կիրառումը խաչասերման օպերատորից
կիրառումից հետո**

2.2.3 Սերնդափոխության խորություն

Սերունդափոխության խորությունը սահմանվում է որպես ժառանգների թվի հարաբերակցությունը ծնողական պոպուլյացիայի հզորությանը (քրոմոսոմների թվին): Օրինակ, եթե այն հավասար է 0,9 , ապա նշանակում է, որ նախորդ պոպուլյացիայի քրոմասոմների 90%-ը փոխարինվում է նոր պոպուլյացիայից լավագույն հայտնաբերված քրոմոսոմների 90%-ով, մինչդեռ հին պոպուլյացիայից նոր պոպուլյացիայի մեջ պահպանվում է լավագույն հայտնաբերված քրոմոսոմների 10%-ը: Այս պարամետրը նույնպես կարող է կարգաբերվել, յուրաքանչյուր սերունդից հետո նվազման միտումով: Օրինակ առաջին սերնդափոխության ժամանակ այն կարելի է ընդունել մեծ (օրինակ՝ 0,9), իսկ վերջին սերունդներում այն իջեցնել (օրինակ 0,1):

2.2.4 Սերնդի ընտրություն

Գենետիկական ալգորիթմում (GA) ընտրության օպերատորը առանցքային դեր է

խաղում որոշելու, թե որ անհատները (լուծումները) են ընտրվում իրենց գեները (տեղեկատվությունը) հաջորդ սերնդին փոխանցելու համար: Գաղափարն այն է, որ նախապատվությունը տրվի ավելի հարմար անհատներին, բայց նաև պահպանի բազմազանությունը՝ երբեմն թույլ տալով ընտրել ավելի քիչ պիտանի անհատների: Ընտրության նպատակը՝

1. Ընտրել ծնողներին վերարտադրության համար (խաչասերում և մուտացիա):
2. Ընտրության ժամանակ կողմնակալություն դրսևորել ավելի հարմար անհատների նկատմամբ:
3. Խուսափել վաղաժամ կոնվերգենցիայից:

Կարելի է առանձնացնել սելեկցիոն օպերատորների հետևյալ տեսակները, կարգային մեթոդ, ռուլետկայի մեթոդ(roulette-wheel selection), մրցաշարային մեթոդ:

1) Կարգային մեթոդ

Այս դեպքում, սկզբում կատարվում է ֆիթնես ֆունկցիայի հաշվարկ, պոպուլյացիայի բոլոր քրոմոսոմների համար, որից հետո կատարվում է քրոմոսոմների կարգաբերում ըստ քրոմոսոմների որոկի անկման: Այնուհետև քրոմոսոմի ընտրության հավանականությունը հաշվվում է հետևյալ բանաձևով՝

$$P_i = \text{rank}(i)/N * (N - 1)$$

որտեղ P_i -ն i -րդ քրոմոսոմի ընտրության հավանականությունն է, N -քրոմոսոմների թիվն է պոպուլյացիայում, $\text{rank}(i)$ - i -րդ քրոմոսոմի դիրքի համարն է կարգավորված ամբողջության մեջ:

2) Ռուլետկայի մեթոդ

Այս դեպքում ընտրությունը կատարվում է ըստ նպատակային ֆունկցիայի արժեքների համամասնության: Քրոմոսոմների ընտրությունը կատարվում է օգտագործելով ռուլետկա անիվի ու «վազքը»: Ռուլետկա անիվը կազմված է N սեկտորներից, որոնց մակերեսները համամասնական են քրոմոսոմների ընտրության հավանականություններին: Անիվի i -րդ սեկտորի չափը համամասնական է i -րդ քրոմոսոմի ընտրության հավանականությանը և որոշվում է հետևյալ բանաձևով՝

$$P_i = \frac{F_i}{\sum_{i=1}^N F_i}$$

որտեղ F_i - ֆիթնես ֆունկցիայի արժեքն է i -րդ քրոմոսոմի համար: Նման ընտրության դեպքում ավելի բարձր հարմարվողականություն ունեցող պոպուլյացիայի անդամներն ավելի մեծ հավանականությամբ կընտրվեն, քան ցածր հարմարվողականություն ունեցողները:

3) Մրցաշարային մեթոդ

Այս դեպքում, սկզբում ամբողջ պոպուլյացիայից պատահականության սկզբունքով ձևավորվում է որոշակի ենթաբազմություն (ենթաբազմություններ), որից (որոնցից) էլ ընտրությունը կատարվում է ըստ նպատակային ֆունկցիայի արժեքների: Պոպուլյացիայից առանձնացվող քրոմոսոմների ենթաբազմություններում քրոմոսոմների թիվը վերցվում է ոչ մեծ (2, 3), կամ դրա ընտրությունը թողնվում է օգտագործողին: Եթե գործ ունենք քրոմոսոմների մեկից ավել ենթաբազմությունների հետ, ապա դրանցից յուրաքանչյուրից ընտրվում է լավագույն քրոմոսոմը, որոնք էլ կազմում են նոր ենթաբազմություն և այս գործընթացը շարունակվում է, մինչև մնա մեկ կամ երկու քրոմոսոմ, որոնք էլ հանդես կգան որպես ծնողներ:

2.2.4 Համապատասխանության ֆունկցիա (ֆիտնես ֆունկցիա)

Գենետիկական ալգորիթմում համապատասխանության ֆունկցիան (fitness function) այն ֆունկցիան է, որը գնահատում է յուրաքանչյուր լուծման (քրոմոսոմի) որակը: Այս ֆունկցիան յուրաքանչյուր անհատին հատկացնում է հարմարության գնահատական, որը ցույց է տալիս՝ որքանով է այդ լուծումը մոտ խնդրի լավագույն տարբերակին: Ալգորիթմի ընթացքում ավելի բարձր հարմարվողականության գնահատական ունեցող անհատները ավելի շատ հնարավորություն ունեն ընտրվելու հաջորդ սերնդի ձևավորման համար (համադրություն և մուտացիա), իսկ ցածր գնահատական ունեցողները կամ դուրս են մնում, կամ ենթարկվում են փոփոխությունների: Այս ֆունկցիան է ղեկավարում ամբողջ որոնման գործընթացը՝

մոտեցնելով ալգորիթմը լավ լուծմանը: Հարմարության ֆունկցիայի ճիշտ ընտրությունը շատ մեծ ազդեցություն ունի՝ թե որքան արագ ու արդյունավետ գենետիկական ալգորիթմը կգտնի լավագույն լուծումը:

Գենետիկական ալգորիթմում հարմարողականության (fitness) ֆունկցիան պետք է բավարարի մի քանի կարևոր չափանիշների, որպեսզի ալգորիթմը արդյունավետ աշխատի:

1. Այն պետք է լինի պարզ և արագ հաշվարկվող, քանի որ ամեն սերնդի բազմաթիվ անհատների համար այդ ֆունկցիան հաշվարկվելու է, և եթե այն ծանր լինի, ամբողջ ալգորիթմի աշխատանքը կդանդաղի:
2. Ֆունկցիան պետք է օբյեկտիվ արտացոլի, թե տվյալ անհատը որքանով է մոտ լավագույն հնարավոր լուծմանը: Այսինքն՝ լավ լուծումները պետք է ունենան բարձր հարմարողականության գնահատական, իսկ վատերը՝ ցածր:
3. Ֆունկցիան պետք է ունենա տարբերակիչ կարողություն, այսինքն՝ տարբեր անհատների միջև հստակ տարբերություն դնի, որպեսզի հնարավոր լինի ընտրել լավագույններին հաջորդ սերնդի համար:
4. Չորրորդ կարևոր հատկանիշն է հավասարակշռվածությունը: Նպատակահարմար չէ, որ մի քանի անհատ ունենան շատ բարձր գնահատական, իսկ մնացածները՝ գրեթե զրո, որովհետև այդ դեպքում ալգորիթմը կարող է շատ շուտ կենտրոնանալ մի հատվածում և կորցնել լուծման ավելի լավ հնարավորությունները:
5. Վերջապես, հարմարողականության ֆունկցիան պետք է անմիջականորեն կապ ունենա խնդրի նպատակի հետ, այսինքն՝ ֆիթնեսի արժեքը պետք է արտացոլի հենց այն չափանիշը, որն ուզում ենք օպտիմալացնել:

2.2.5 Կանգառի պայմանը

Գենետիկ ալգորիթմում կանգառի պայմանը որոշում է, թե երբ պետք է ավարտել ալգորիթմի աշխատանքը: Կանգառի պայմանը շատ կարևոր է, որովհետև եթե

ալգորիթմը շատ շուտ կանգնի, հնարավոր է՝ չգտնի լավ լուծում, իսկ եթե շատ երկար շարունակվի, կարող է անիմաստ շատ ռեսուրս ծախսել: Սովորաբար օգտագործվում են մի քանի տարբեր կանգառի պայմաններ: Առաջին տարբերակը՝ երբ ֆիքսված քանակով սերունդներ են անցնում, օրինակ՝ եթե սահմանվել է 500 սերունդ, ապա 500-րդ սերնդից հետո ալգորիթմը կանգնում է: Երկրորդ տարբերակը՝ երբ հասնում է նախապես սահմանված լավ ֆիթնես արժեքի, այսինքն՝ եթե անհատներից մեկի հարմարողականության գնահատականը հասնում է, ասենք, 0.99-ի կամ 100-ի, ալգորիթմը համարում է, որ լավ լուծում է գտնվել ու դադարում է: Երրորդ տարբերակը՝ երբ սերունդների ընթացքում ֆիթնեսի արժեքը այլևս չի բարելավվում, օրինակ՝ վերջին 50 սերնդի ընթացքում լավագույն ֆիթնես արժեքը նույնն է մնում, այսինքն՝ ալգորիթմը կարծես կանգ է առել մի կետի վրա ու առաջ չի շարժվում: Երբեմն էլ օգտագործվում է ժամանակային սահմանափակում, այսինքն՝ ալգորիթմը կանգնում է, երբ անցնում է որոշված ժամանակը, օրինակ՝ 5 րոպե:

Գրականության ցանկ

- 1) [MING. X. (2009) ANALYTIC METHODS FOR THE FPGA PLACEMENT PROBLEM.]
- 2) [Waleed Bin Owais, Iyad W. J. Alkhazendar, and Dr. Mohammad Saleh. Evaluating the impact of different types of crossover and selection methods on the convergence of 0/1 Knapsack using Genetic Algorithm. Dhinaharan Nagamalai et al. (Eds): CSEIT, WiMoNe, NCS, CloT, CMLA, DMSE, NLPD - 2020 pp. 01-16, 2020. CS & IT - CSCP 2020 DOI: 10.5121/csit.2020.101101 ©2020 By AIRCC Publishing Corporation. This article is published under the Creative Commons Attribution (CC BY) license.]
- 3) [<https://slideplayer.com/slide/12678537/>]
- 4) [https://www.researchgate.net/figure/Steps-of-Genetic-Algorithms_fig3_229043321]
- 5) <https://www.geeksforgeeks.org/crossover-in-genetic-algorithm/>